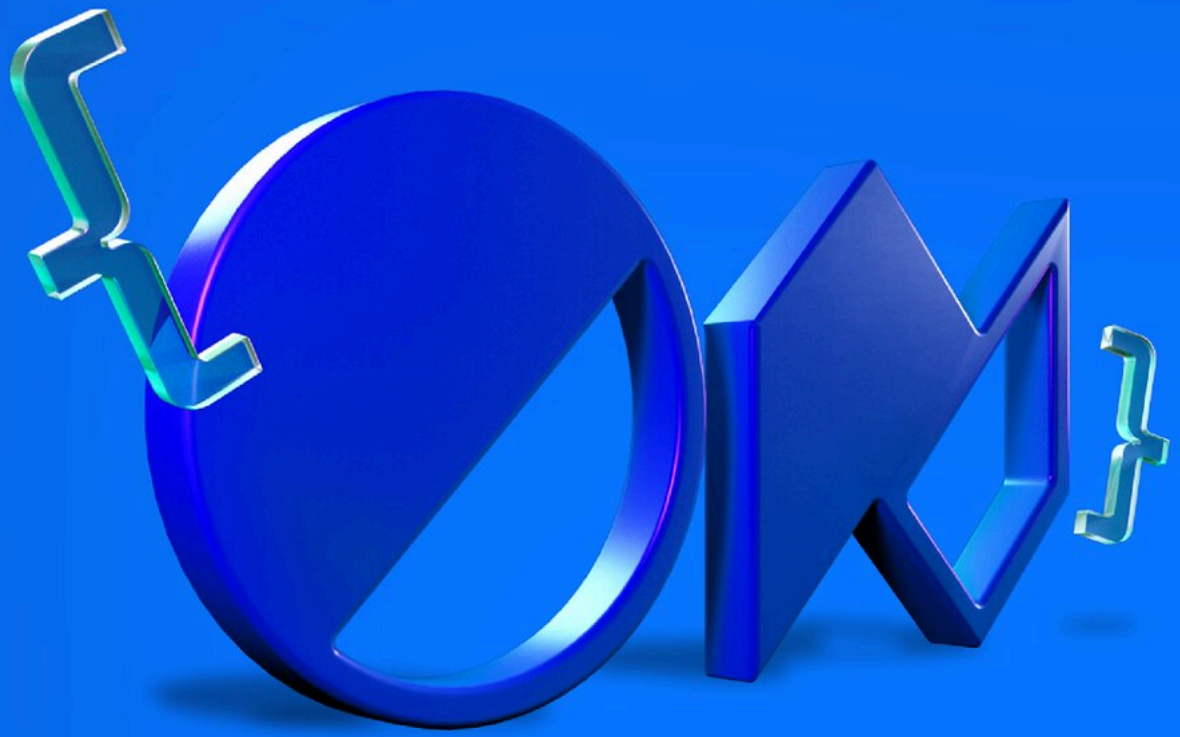
Search is test-time compute

Han Xiao, VP of AI, Elastic

ELASTICON AI SAN FRANCISCO · NOVEMBER 4, 2026

SCAN FOR THE SLIDES

hanxiao.io/elasticon-sf-2026



elastic

Search has mostly improved by training harder.

Bigger encoders, cleaner data, faster indexes, stronger rerankers.



Recent LLMs found a second axis: compute at inference

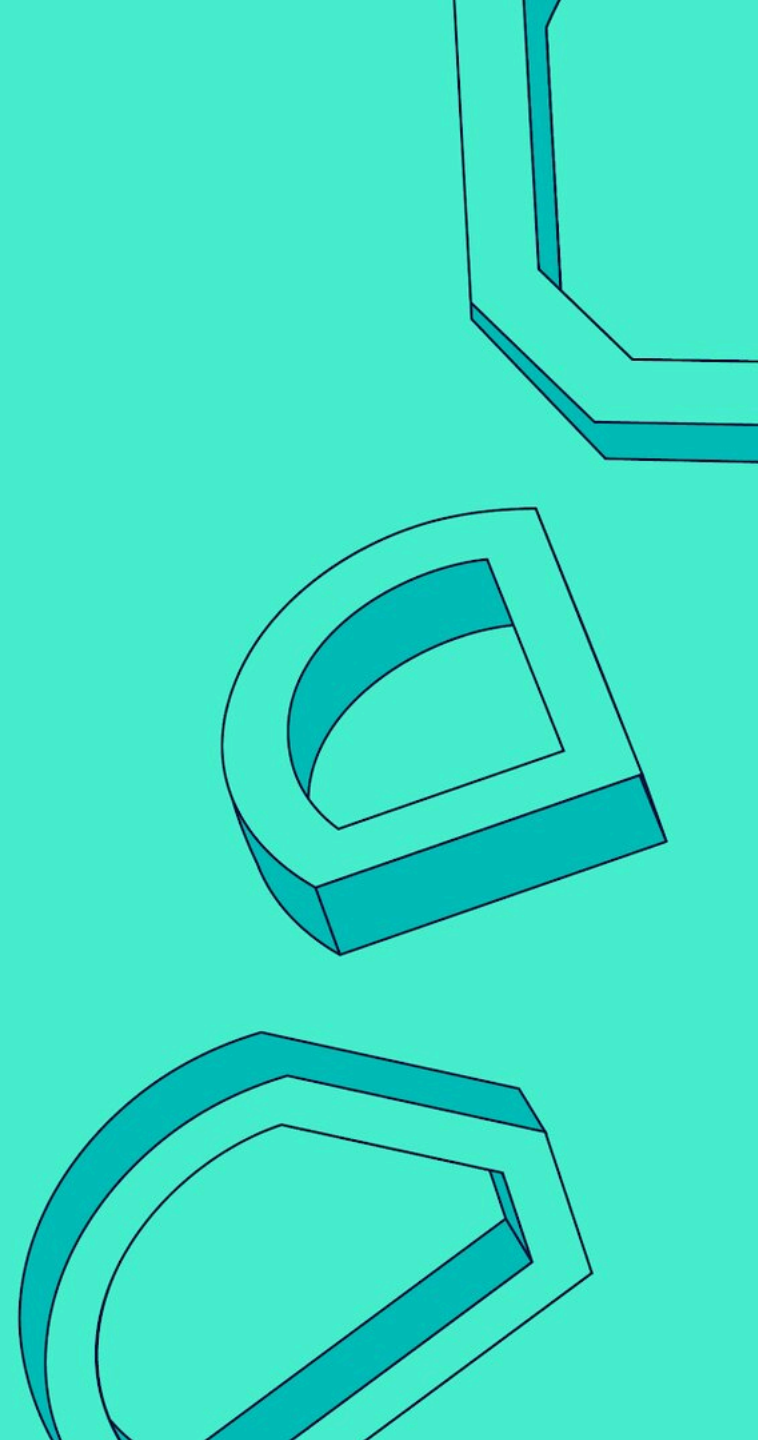
Test-time scaling: rather than a bigger model, the same model does more work per query, along a smooth cost-quality curve.



> Having a bot think for just **20 seconds** in a hand of poker got the same performance boost as scaling up the model by **100,000x** and training it for 100,000 times longer.
>
> Noam Brown, OpenAI, 2024

Fun fact: search has always lived on that axis.

Embed, retrieve, rerank, expand, fuse. None of it happens at training time. All of it runs after the user presses Enter.



Inference time is a currency, not a KPI

UNTIL NOW: A KPI

Inference time is a KPI

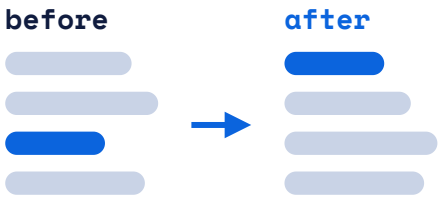
A property of search: latency written into an SLA, a number to stay under. Exceed it and you cannot ship. **It never takes part in a trade.** To get more accurate, you can only go back and swap in a bigger model.

REWRITTEN AS →

IN THIS TALK: A CURRENCY

Inference time is a currency

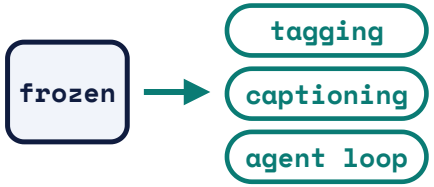
Keep the model, keep the weights, and raise the budget from 10 ms to 100 ms. What does it buy?



BUYS

Relevance

The same task, ranked better.



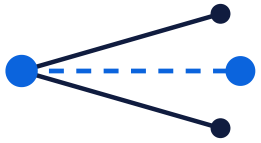
BUYS

A new capability

Tasks it was never trained for.

Four demos, all training-free

Each one gets better performance or a new capability just by spending more compute at test time.



01

Programs that transfer

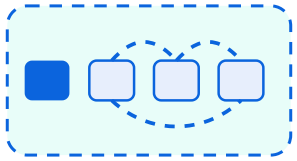
An LLM writes retrieval programs over one frozen encoder. They transfer to model families they never saw.



02

Image tags for free

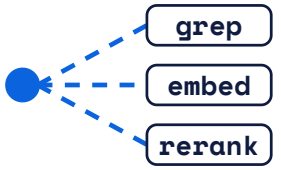
A frozen retrieval model becomes an open-vocabulary image tagger.



03

One call, no index

One listwise reranker call over a whole page replaces chunking, embedding and indexing.



04

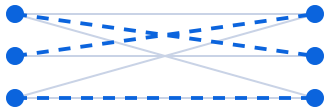
Agent-built pipelines

An agent assembles its own search pipeline from local tools, per query.

Three ways to spend compute on a frozen model

The weights are the same in all three. They differ only in what enters the computation.

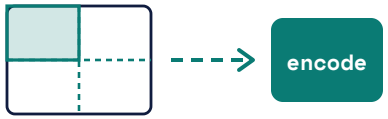
A DEEPER PASS



Read more of the pass you already ran: every token or patch vector, not only the pooled one.

New information: recovered from inside the pass

B MORE PASSES



Run the encoder again on new views: split the document, crop the image, re-encode.

New information: taken from the input

C CALIBRATION



Transform the vectors you already have: whitening, label propagation, optimal transport.

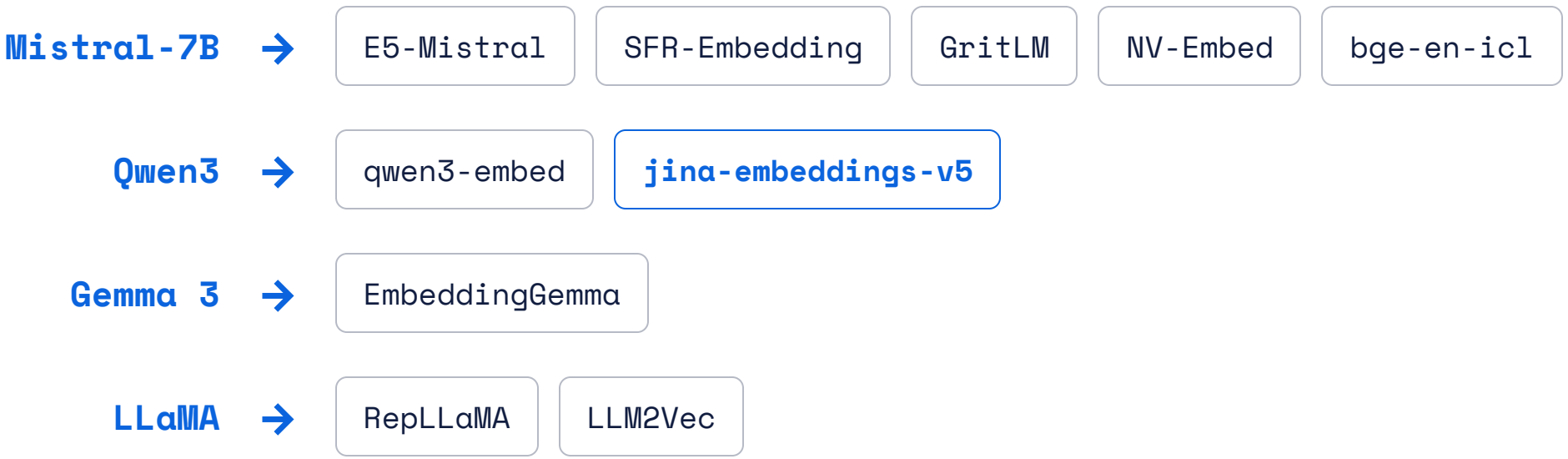
New information: none

01

Programs on a frozen encoder

Today's small embedders are distilled from LLMs

Test-time compute is said to belong to large reasoning models. If it lives in the LLM representation space, distilled encoders might inherit it.



> We first asked this in December 2024: **jina.ai/news/scaling-test-time-compute-for-embedding-models**

Scoring runs from one cosine to late interaction

A single-vector encoder can still read a document more than once: split it into sentences and keep the best match.

SINGLE-VECTOR COSINE

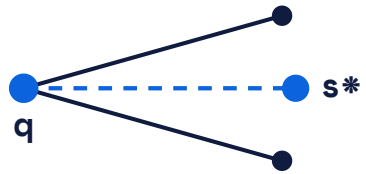


$$\cos(q, d)$$

one vector per document

FROZEN BASELINE

SENTENCE MAXSIM

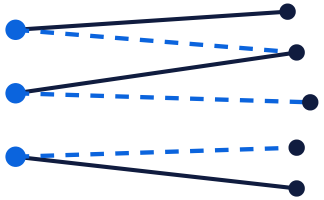


$$\max_s \cos(q, s)$$

same encoder, document split into sentence vectors

TEST-TIME COMPUTE

COLBERT MULTI-VECTOR



$$\sum_i \max_j \cos(q_i, d_j)$$

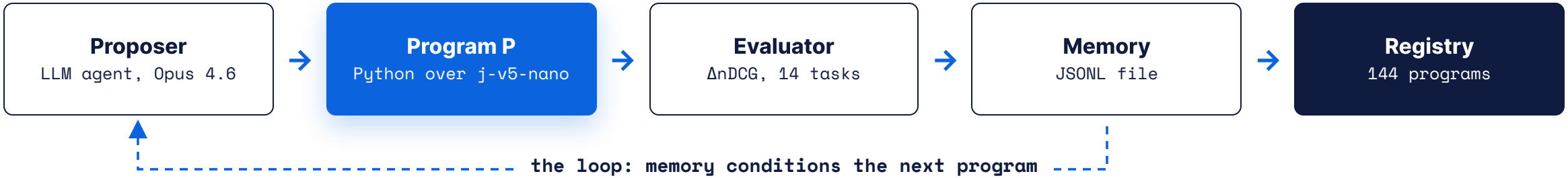
every query token, max over all document tokens

NEEDS A MULTI-VECTOR MODEL

Rules: no external LLM in the query path (HyDE, Query2doc), no second retriever (GQR), no learned parameters (MetaEmbed). One frozen encoder API, and one **embed_fn** call is one unit of compute.

An LLM agent writes programs over the frozen encoder

Autoresearch: change one program, run a fixed-budget evaluation, keep it if the metric improves, otherwise revert.



WRITES	THE ENCODER	SCORES ON	LOGS	AFTER 144 ROUNDS
1 program per generation	239M, frozen small + fast	14 MMTEB tasks ΔnDCG@10	Δ, cost, parent, lesson per program	144 programs scored

Hyperparameters, task-specific routing, external models and learned parameters are rejected automatically. **arXiv:2605.11374**

We search on one encoder and test on three others

Neither search ever sees the 19 held-out tasks or the two unseen encoder families.

DISCOVERY · THE LOOP SEARCHES HERE

j-v5-nano
239M · Jina
THE DISCOVERY ENCODER

small + fast → quick loop cycles

SEARCHED OVER
14 MMTEB Tier-1 tasks · legal, financial, long-doc, general

→ one program, must generalize

HELD-OUT · THE LOOP NEVER SEES THESE

j-v5-small
568M · Jina
SAME FAMILY

gemma-300m
303M · Gemma 3
UNSEEN FAMILY

qwen3-0.6b
600M · Qwen3
UNSEEN FAMILY

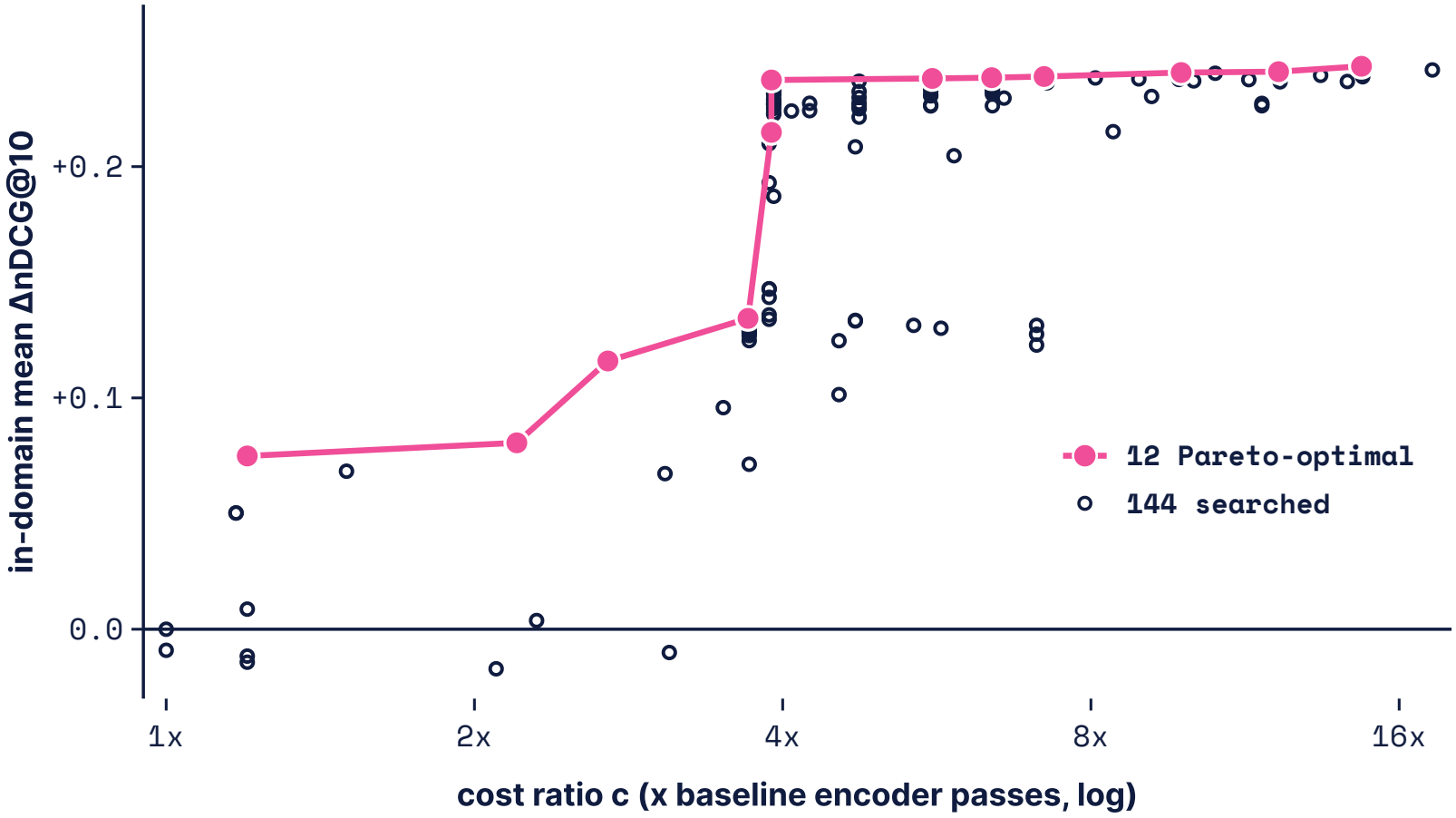
unseen families → the real transfer test

TESTED ON
19 MMTEB Tier-2/3 tasks · none in discovery (summarization, QA, fact-check, ...)

Metric: ΔnDCG@10 against the cosine baseline. **Cost c:** encoder forward passes per query relative to the baseline.

Told to spend compute, the search draws a Pareto curve

Rewarded for in-domain gains and for spending compute, the loop climbs from +0.07 to +0.24 mean ΔnDCG@10 over 144 programs.



144

programs searched, one per generation

12

Pareto-optimal, from c = 1.2 to 14.7

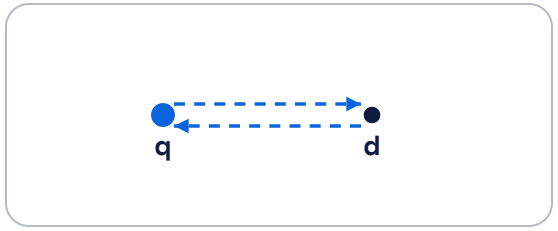
+0.24

best in-domain mean ΔnDCG@10

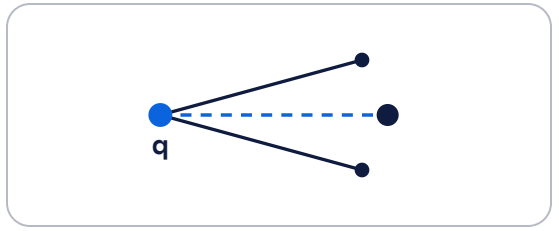
In-search numbers on the 14 discovery tasks. The held-out test comes next.

Twelve programs on the compute frontier

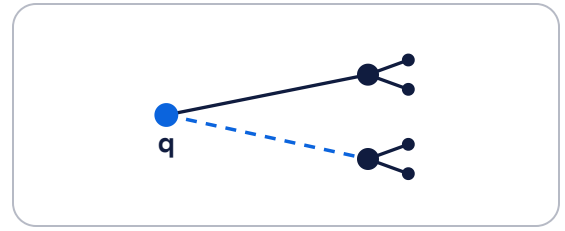
Each is a training-free recombination of the same frozen vectors. Cost rises from 1.2x to 14.7x.



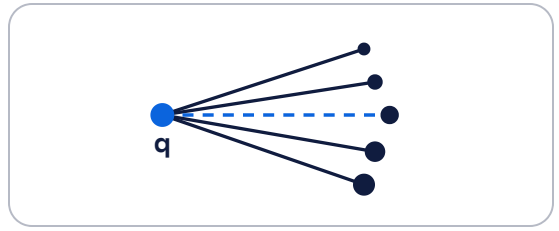
BidirZScore c=1.2



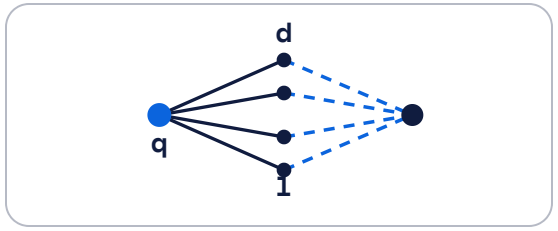
SentMaxSim c=2.2



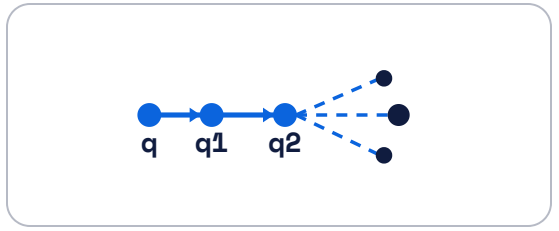
AdaptGranularity c=2.7



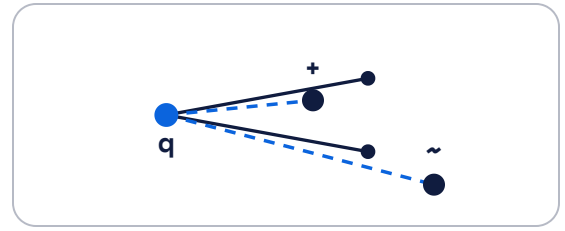
CoverageTriple c=3.7



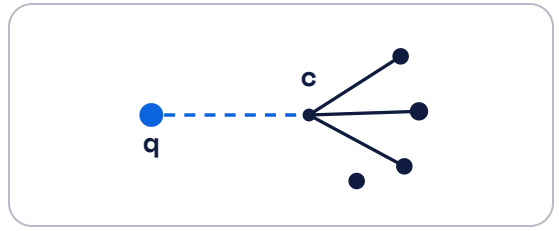
LexicalHybridRRF c=3.9



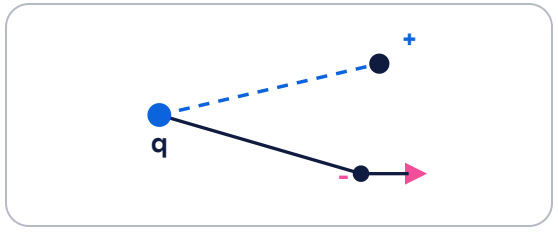
CrossRoundRRF c=3.9



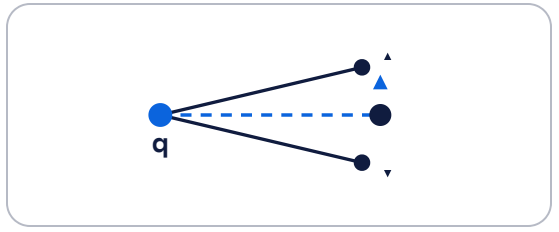
DiverseDualCtx c=5.6



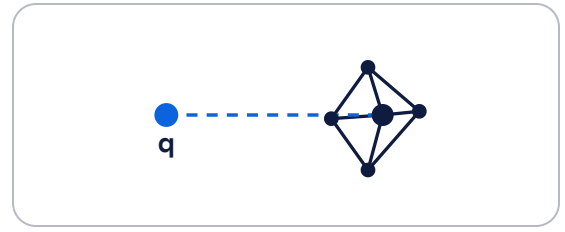
ConsensusRocchio c=6.4



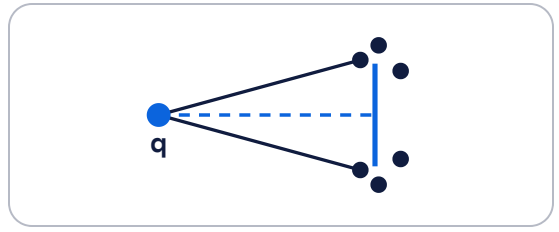
NegContrastive c=7.2



MomentumProg c=9.8



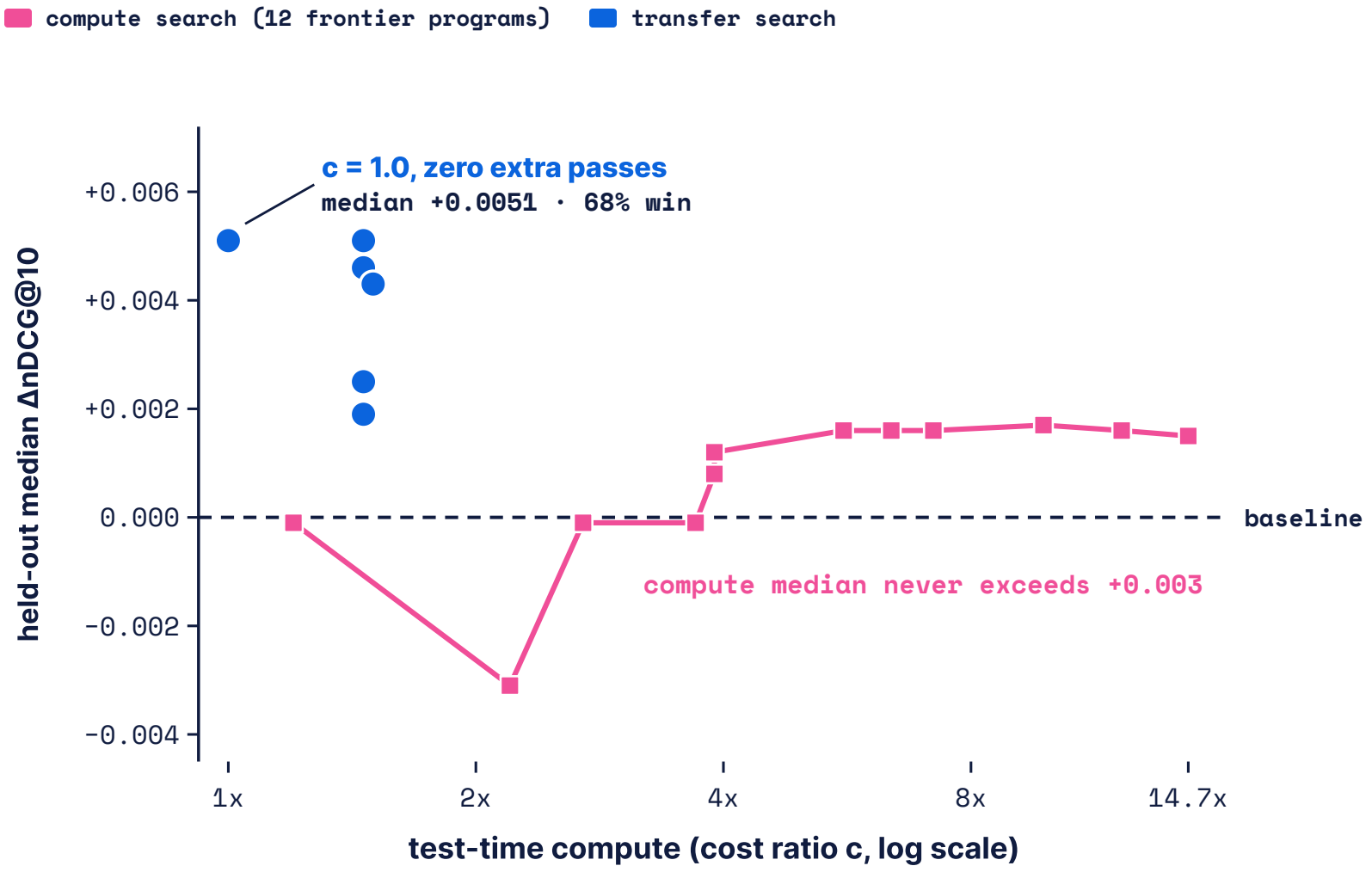
GraphCentrality c=12.2



FisherStability c=14.7

On unseen encoders, compute is flat and structure is not

A program with zero extra forward passes beats the most expensive compute program on the held-out median.



+0.003

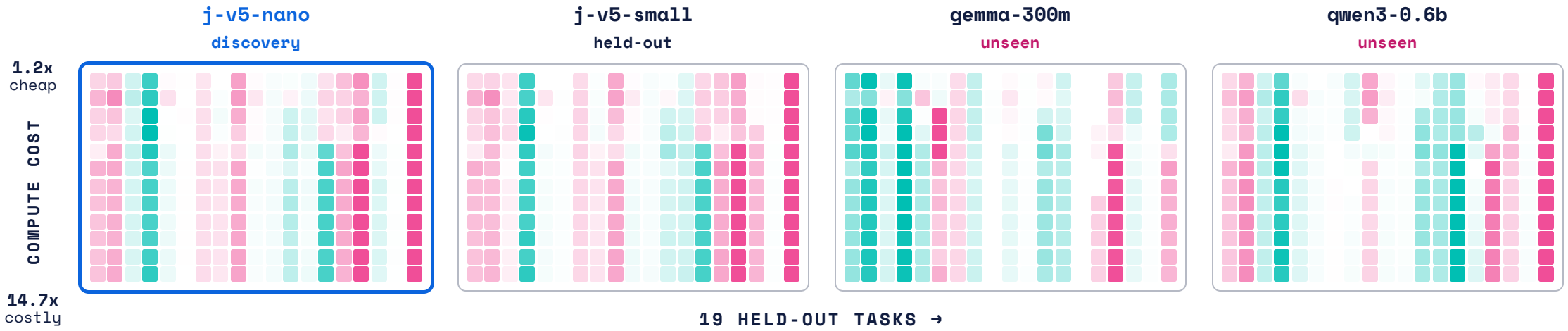
the compute frontier's held-out median never goes above this, all the way to 14.7x

+0.0051

held-out median of a program at c = 1.0, with a 68% win rate

Compute helps about half the cells and collapses the rest

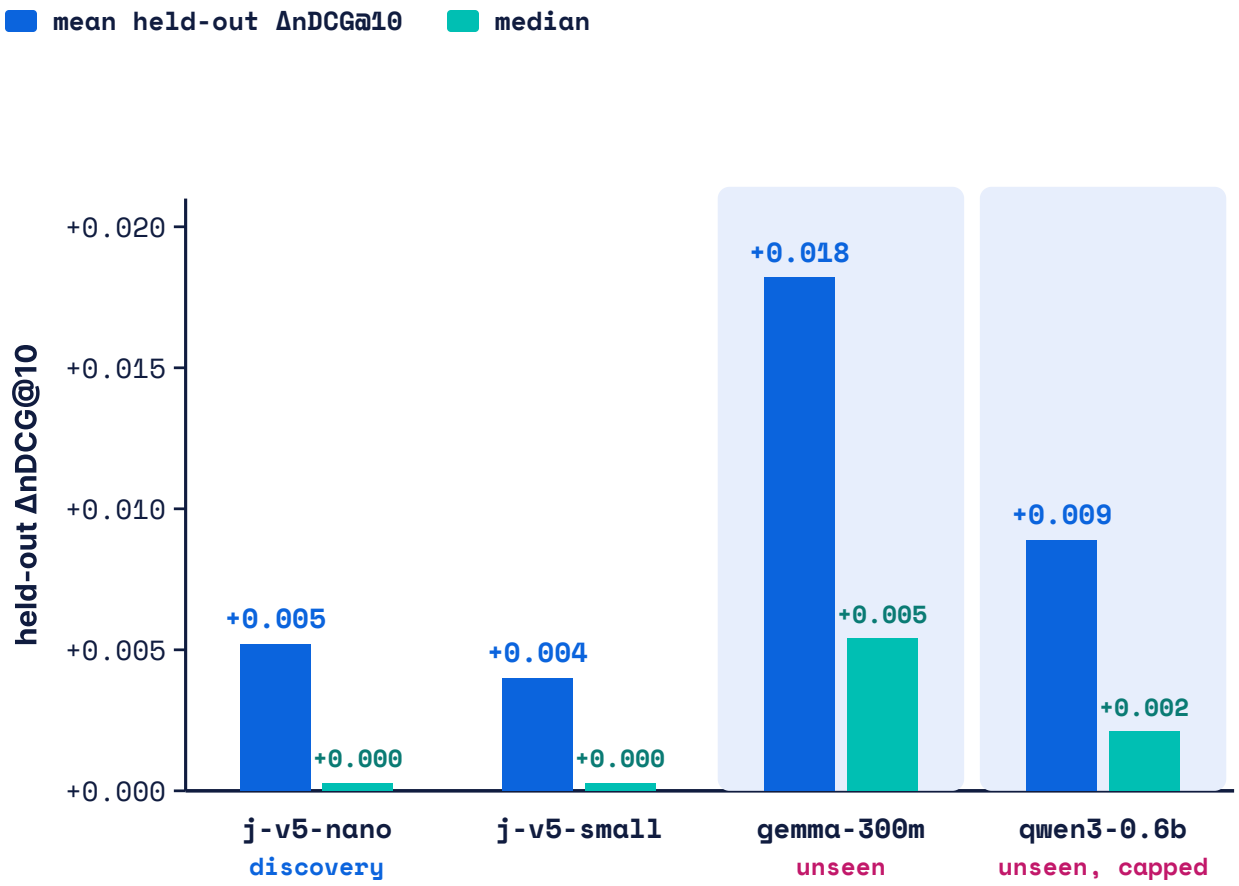
12 programs by cost × 19 held-out tasks × 4 encoders. 485 of 912 cells improve, but the worst falls to −0.98, so the pooled mean is −0.016.



Rows are programs ordered by cost, columns are held-out tasks. 52 of 76 task-encoder pairs improve under at least one program, yet the deep pink cells keep the pooled mean below zero.

A transfer search finds cheap programs that carry over

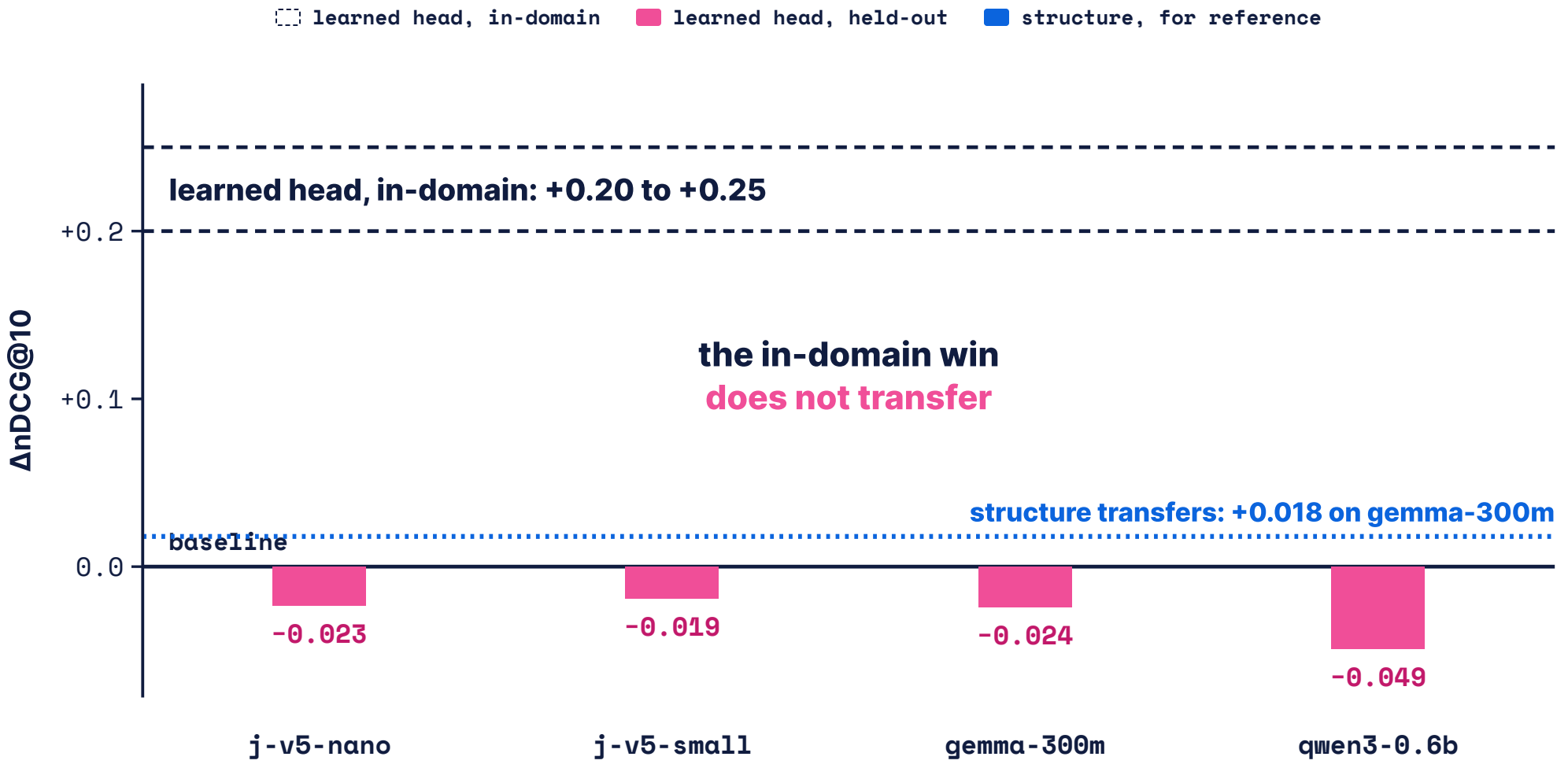
Admit a program only if a held-out validation split improves and no task regresses past 0.05. Six programs pass, all at 1.5x or less.



- Mean gains are positive on all four encoders and largest on the families the search never saw.
- Transpose-Consensus never loses a task: **83% win rate**, worst cell **0.000**.
- Applied unmodified to French and Greek: median **+0.016**, **86%** win rate.

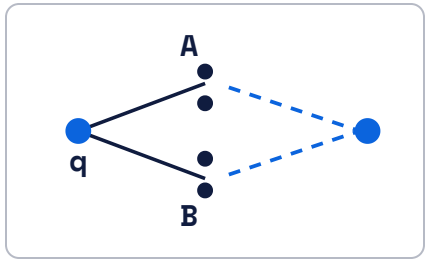
A learned head at the same data budget memorizes

Trained on the 14 discovery tasks, a linear, low-rank or MLP head gains +0.20 to +0.25 in-domain and falls below baseline on every held-out encoder.



The search re-derives classical IR in embedding space

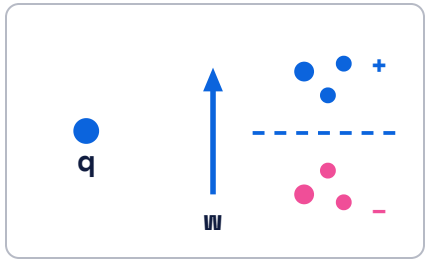
Two methods appear without any seed. Two are operationalized from seeded ideas. All depend on cosine geometry, so they carry across encoders.



Reciprocal Rank Fusion

APPEARED WITHOUT A SEED

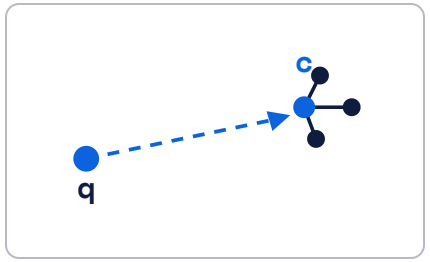
fuse two rankings into one consensus



Fisher Linear Discriminant

APPEARED WITHOUT A SEED

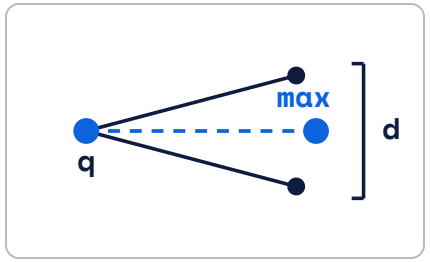
the axis that best separates relevant from not



Rocchio Pseudo-Relevance Feedback

OPERATIONALIZED FROM A SEEDED IDEA

pull the query toward the relevant centroid



Sentence-level MaxSim

OPERATIONALIZED FROM A SEEDED IDEA

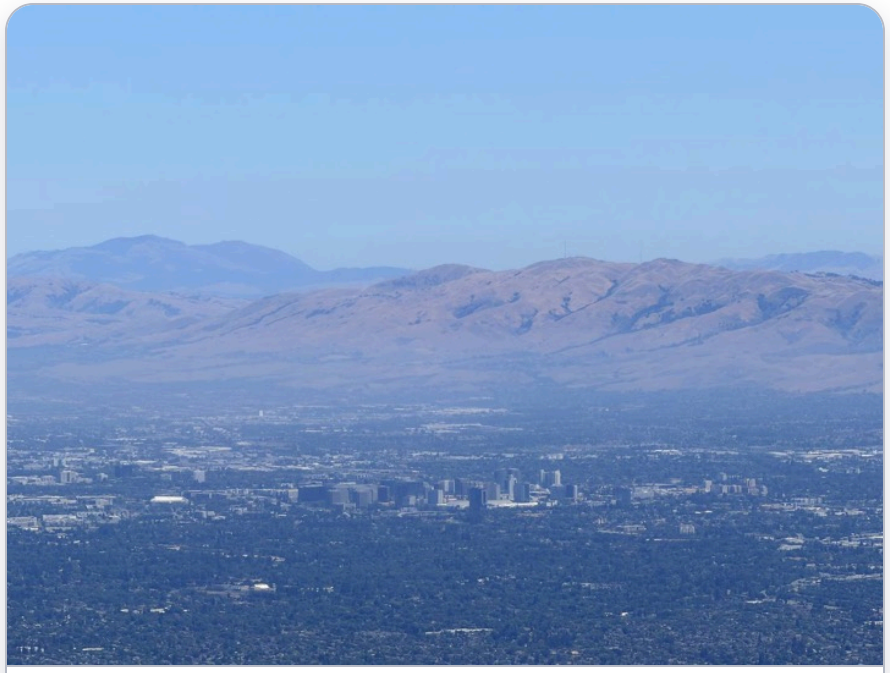
score the best sentence, not the mean

02

A retrieval model tags images

Open-vocabulary tags from a frozen embedding model

No training, no second model, no dictionary, no POS tagger. Every tag below comes from test-time compute alone, verbatim.



mountain · phoenix · sky · rooftop · location



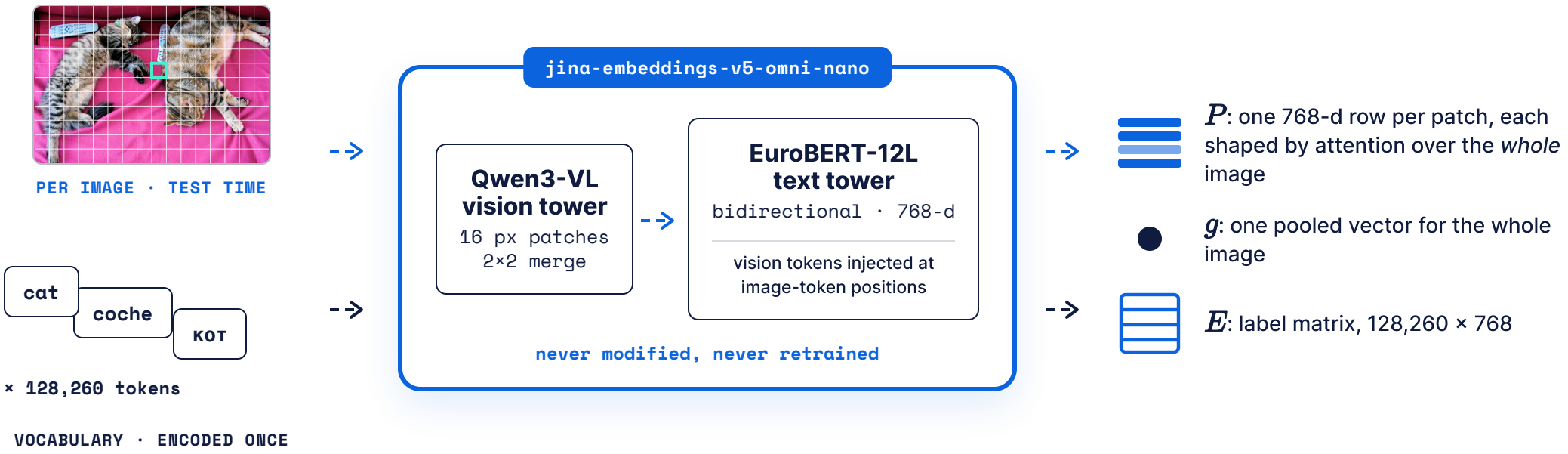


clock · thermostat · dial · dst · wifi

9 photos · top-5 tags each · verbatim

One model, one space for images and text

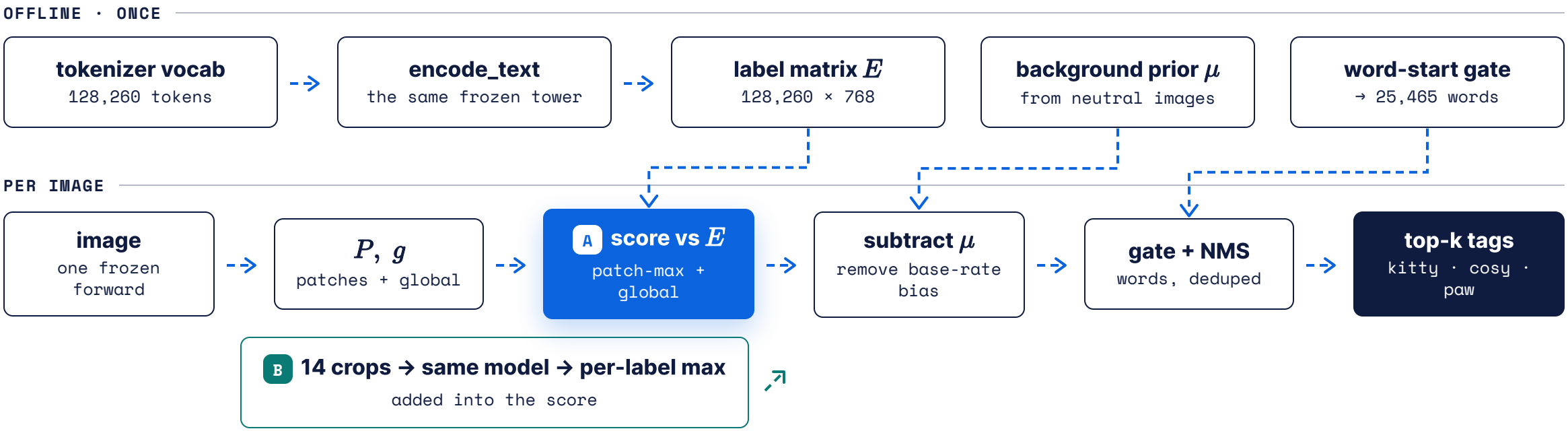
jina-embeddings-v5-omni-nano: about 1B parameters, 768 dimensions. A retrieval encoder with no tagging head, never trained for this task.



Both paths end in the same space, so any patch of any image can be scored against any word the model knows.

The whole tagger is test-time compute

The label set is the tokenizer's own vocabulary: 128,260 tokens encoded once, gated to 25,465 whole words.

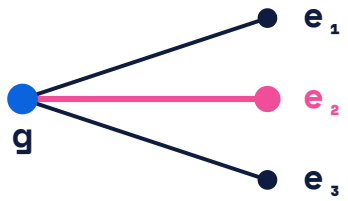


Every box is the frozen model or plain arithmetic: A reads more of one pass, B runs new passes on new pixels.

Score every patch, not only the pooled vector

The patch vectors already exist in the forward pass. Each label keeps its best patch, like ColBERT late interaction. mAP goes from 0.264 to 0.635 for free.

GLOBAL POOLED

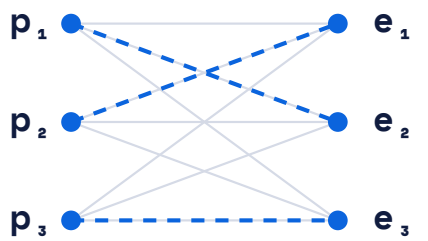


$$s_\ell = \langle g,\, e_\ell \rangle$$

one vector, dominated by the most salient object

mAP 0.264

PER-LABEL MAX OVER PATCHES



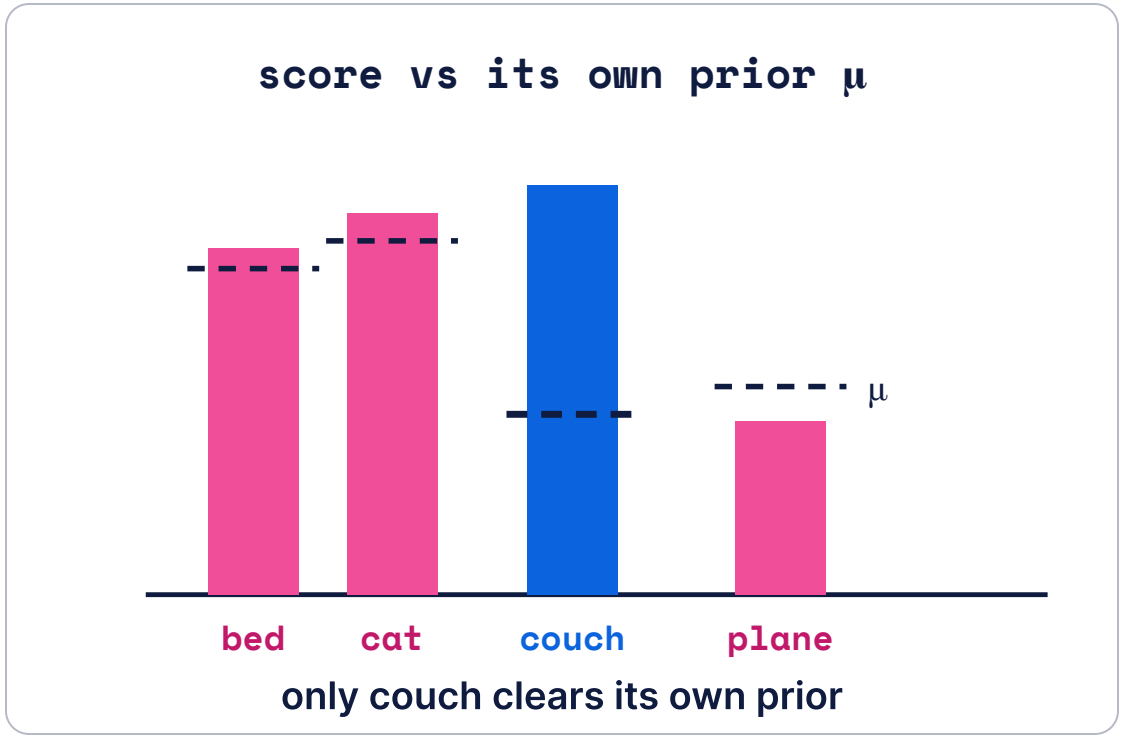
$$s_\ell = 0.7 \max_{p \in P} \langle p, e_\ell \rangle + 0.3 \, \langle g, e_\ell \rangle$$

each label keeps its strongest patch match

mAP 0.635

Each label is compared with its own prior

Generic words such as bed and cat sit close to every image. Subtract a per-label prior estimated once from neutral images.

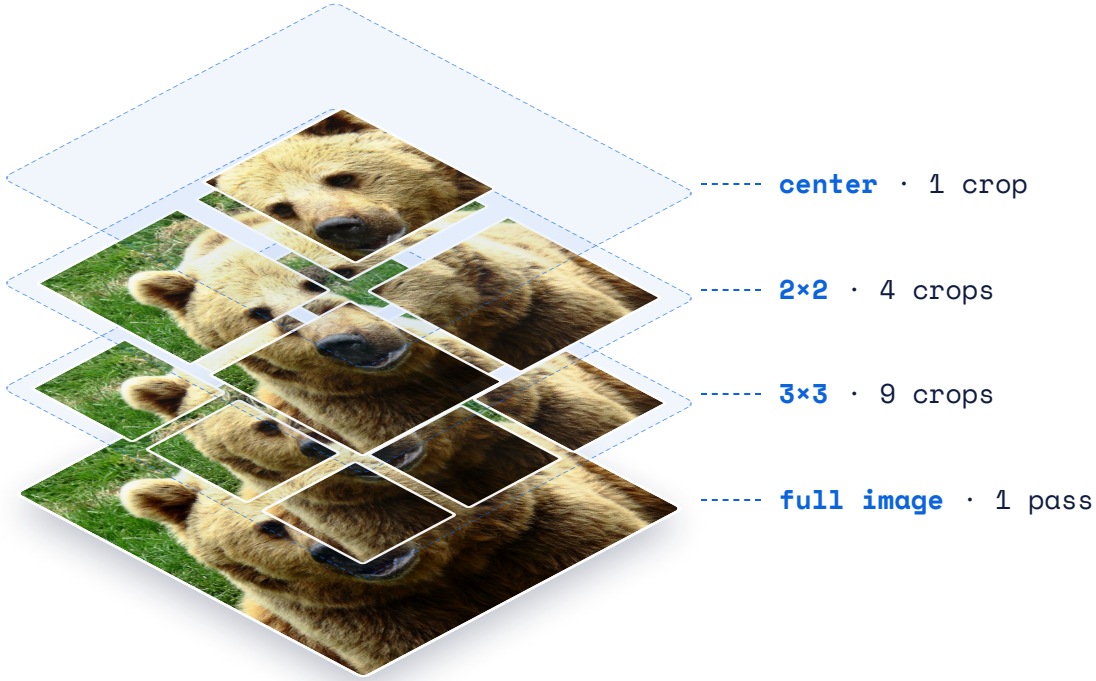




- No calibration set and no labels, one offline pass.
- **Word-start gate:** keep tokens that begin a word, using the tokenizer's own leading-space mark.
- **Embedding-NMS:** drop any word within cosine 0.6 of one already kept, so cat, kitty and кот collapse into one.

Crops give the model new pixels

14 crops through the same frozen model. A small object fills its own crop, so weak evidence becomes strong. Per-label max works, averaging hurts.

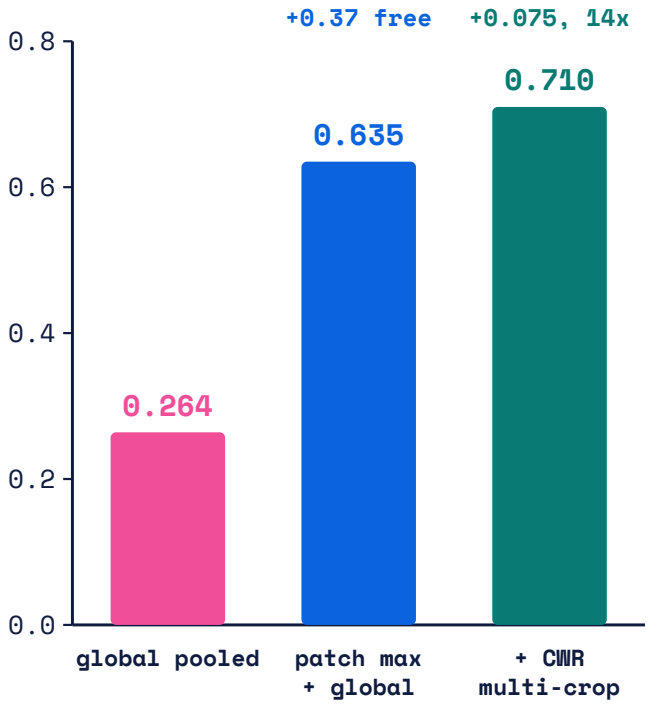


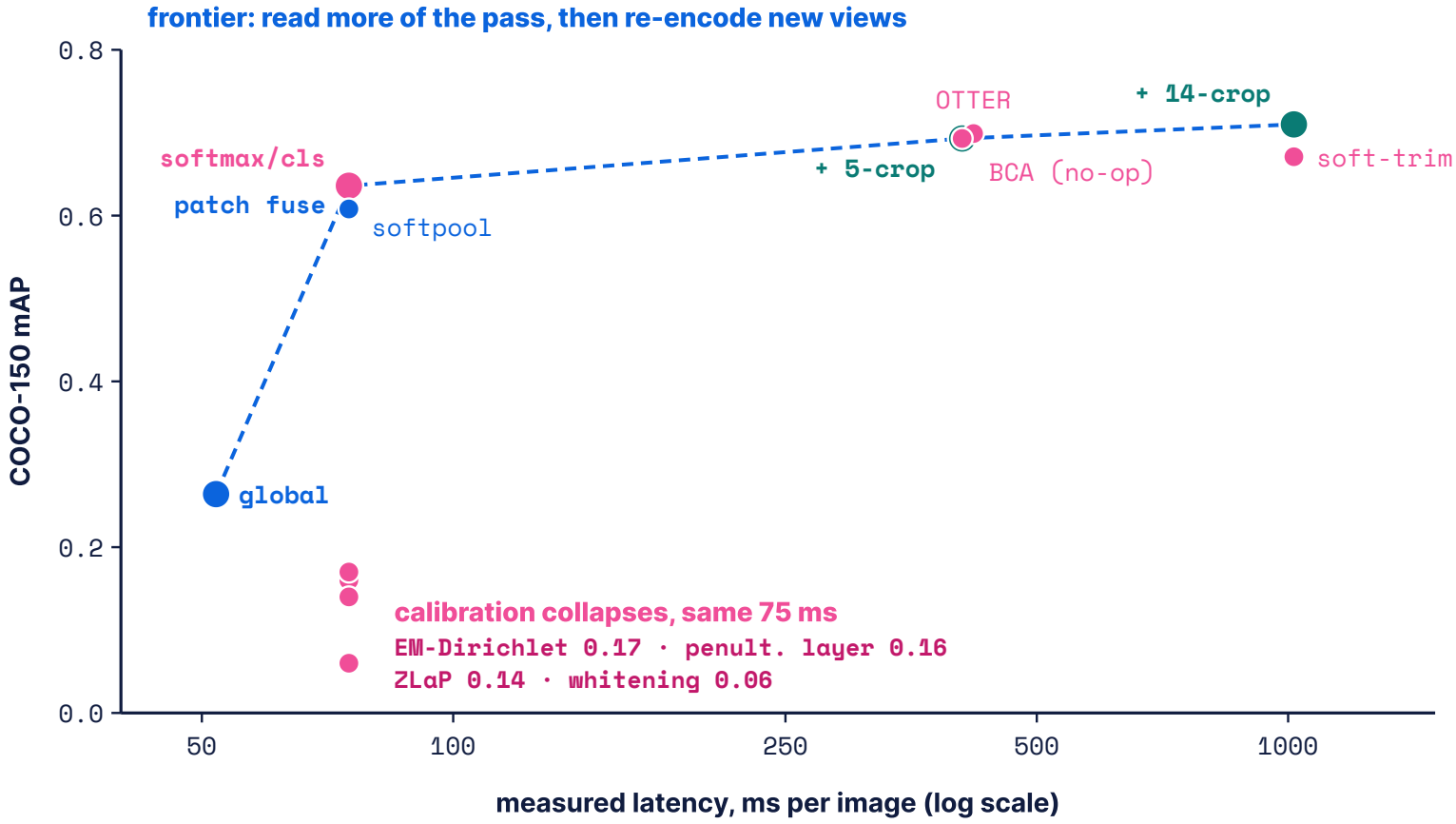
1 pass	wolf	roar	muzzle
+ 14 crops · per-label max	fur	bear	muzzle

- **+0.075 mAP** for 14x the passes: 0.635 to 0.710.
- P@1 goes from **0.753** to **0.813**.
- One pass reads **wolf, roar, muzzle**. With crops it reads **fur, bear, muzzle**.

Accuracy climbs with each kind of new information

Reading more of the pass gives the big step. Re-encoding new views gives the next one, at higher latency.

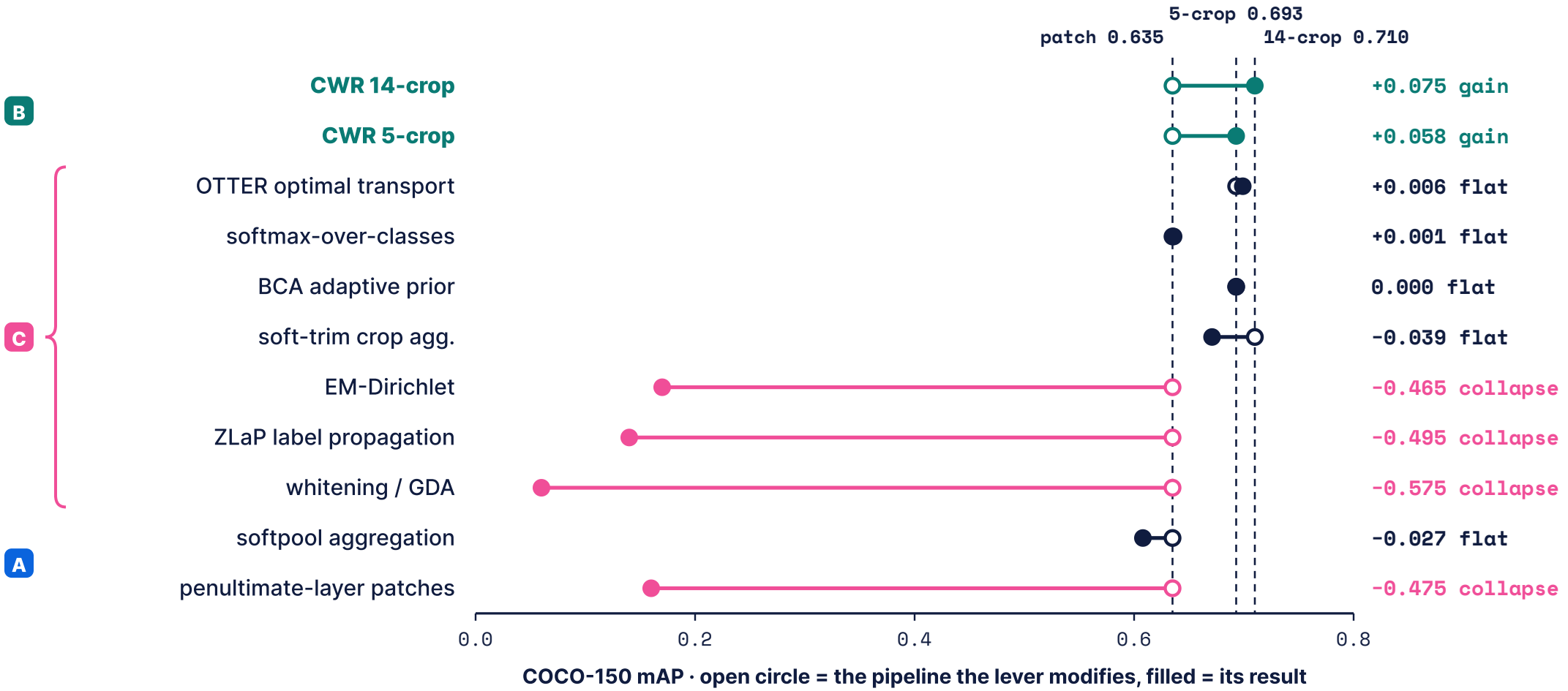




COCO-150 mAP: global → patch → + 14 crops

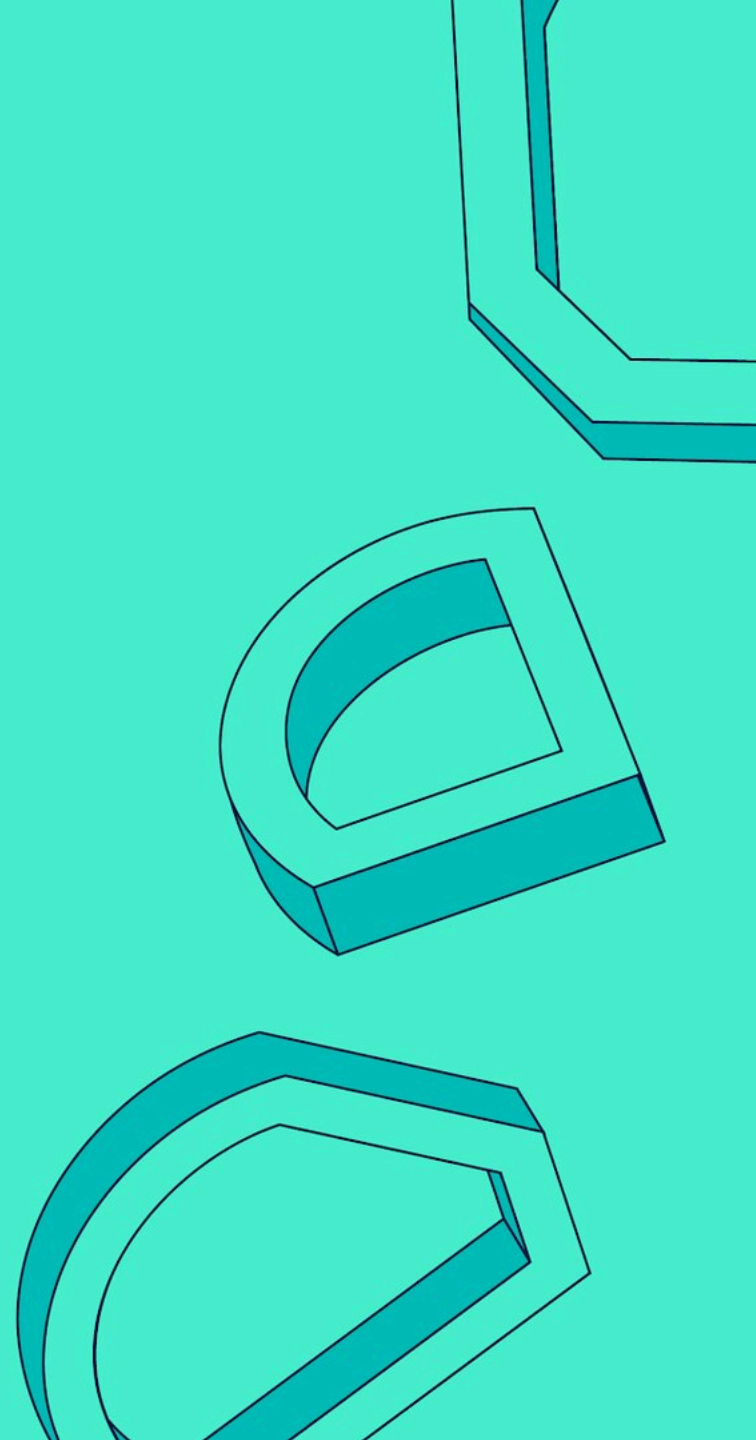
Calibration does not move accuracy

The 2024 to 2026 training-free literature bets on calibration. On this encoder every calibration lever is flat or collapses.



Only new information moves accuracy.

Calibration rearranges what the model has already seen.
Patches and crops give it more to look at.



It ships on device in Omni

Tagging reuses the patch rows of the embedding forward pass. Tags become search snippets for images, video segments and scanned PDFs.

~0.6 ms

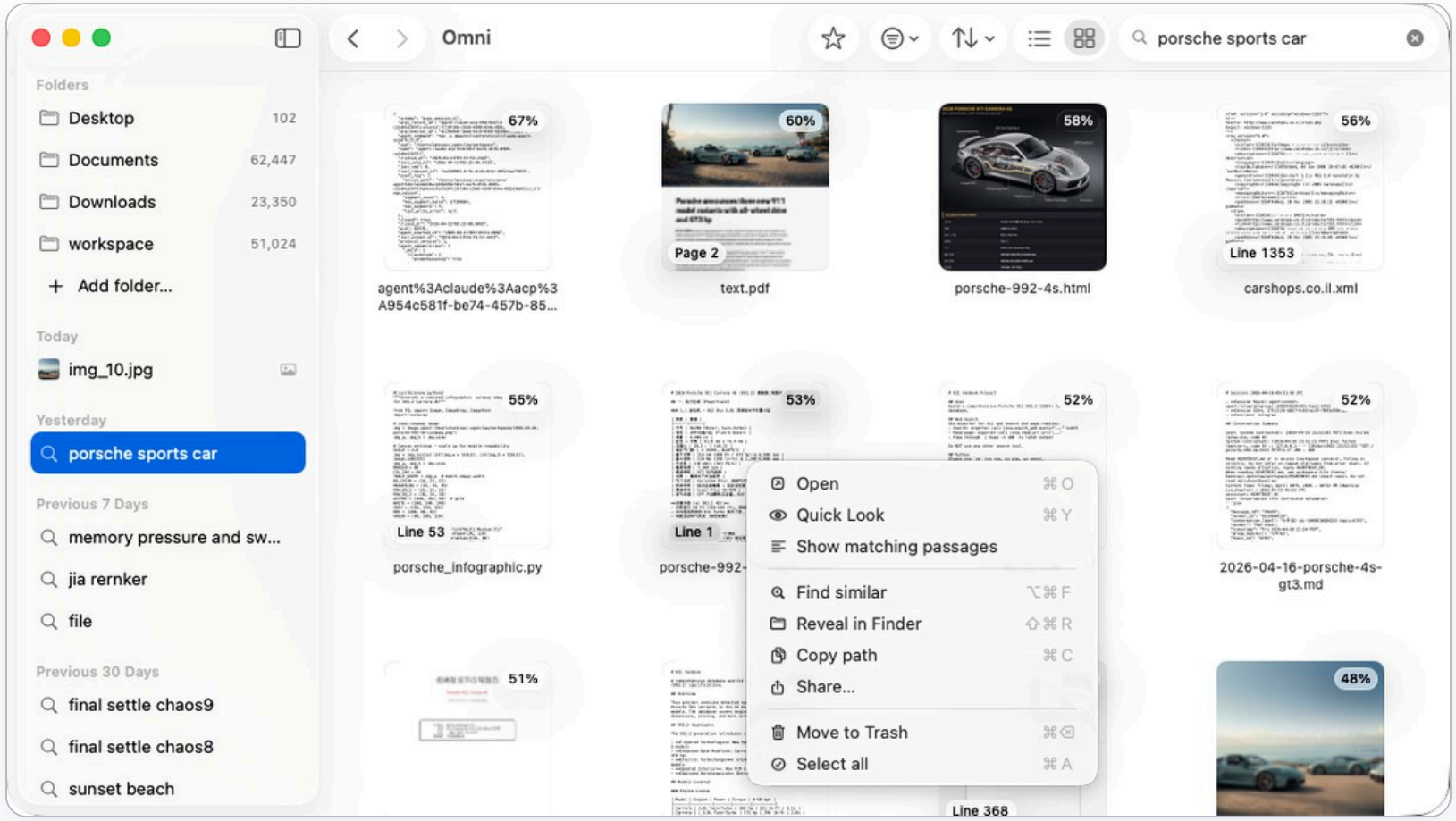
per image

~4%

of the embed step

64

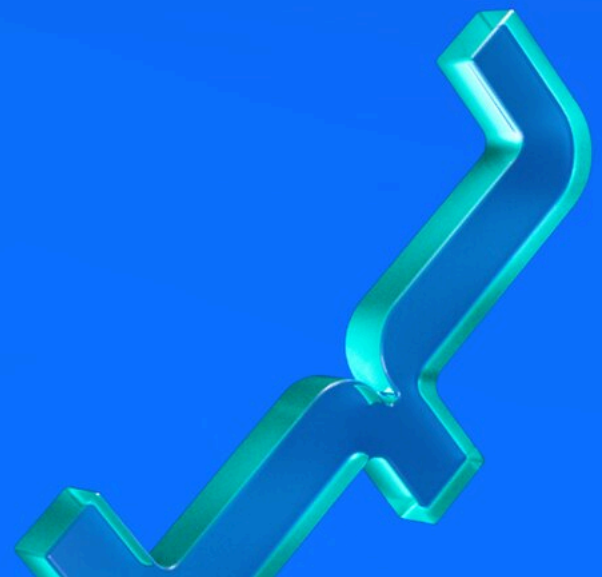
images calibrate the prior on device



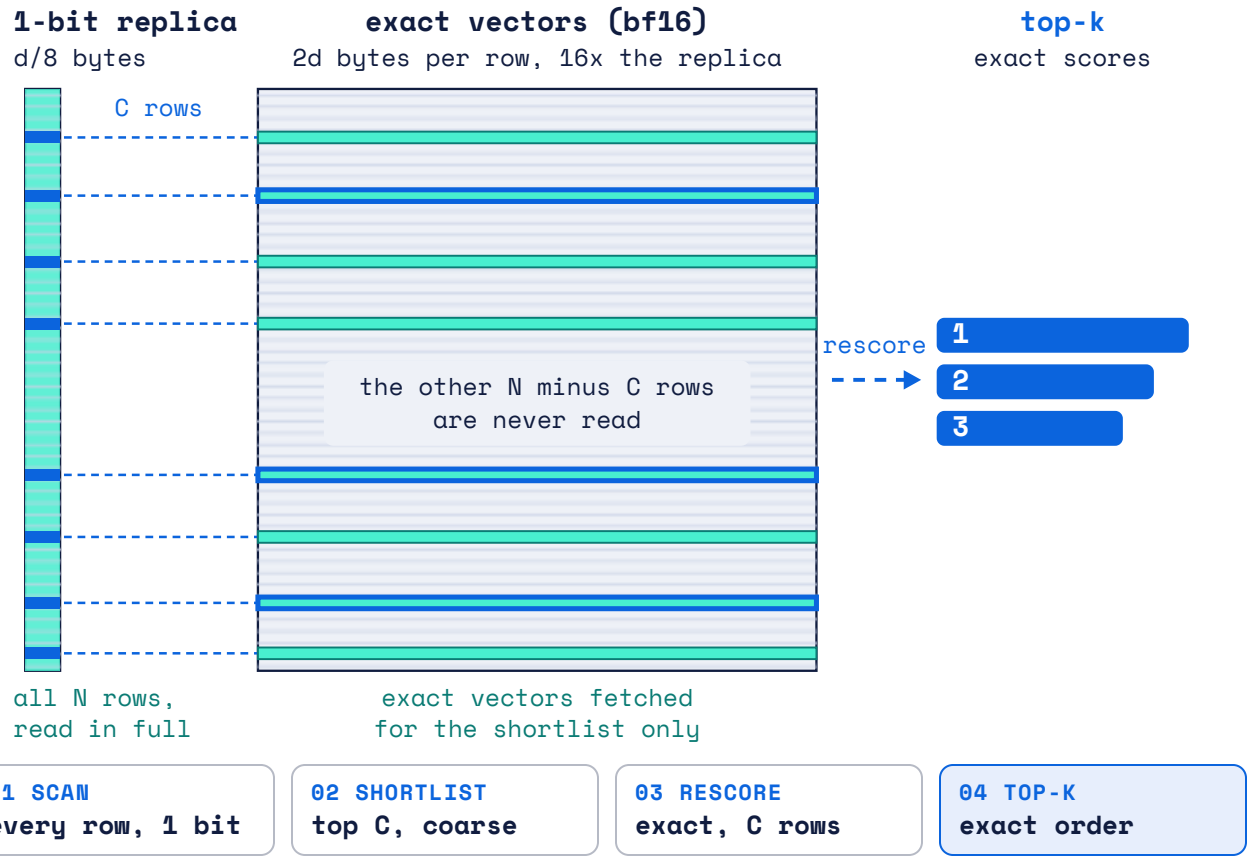


03

One call replaces an index



Spend compute on rescoring, not on building an index



01 SCAN	02 SHORTLIST	03 RESCORE	04 TOP-K
every row, 1 bit	top C, coarse	exact, C rows	exact order

The replica is one sixteenth the width of the exact vectors; teal is the part one query actually reads.

Omni has no ANN index, so every query scans the whole corpus. A graph index needs 256 bytes of links per row: 0.95 GB at four million chunks, **close to three times the matrix a query scans**, and a continuously edited corpus keeps invalidating it.

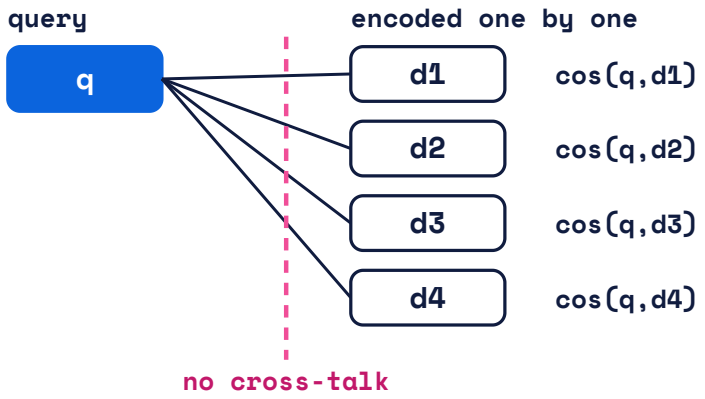
So the same space is stored twice: **the scan reads a 1-bit replica, and exact vectors are touched only for the shortlist**. The coarse score only decides who enters the shortlist; it does not need to be accurate, only wide enough. **The final order is set by exact rescoring.** Candidate set: `min(4096, max(1024, topK×32))`.

Let the candidates see each other in one forward pass

jina-reranker-v3 (0.6B) reads the query and up to 64 documents together. BEIR nDCG@10 61.94, against 56.51 for bge-reranker-v2-m3 and 56.28 for Qwen3-Reranker-0.6B at the same size.

Bi-encoder

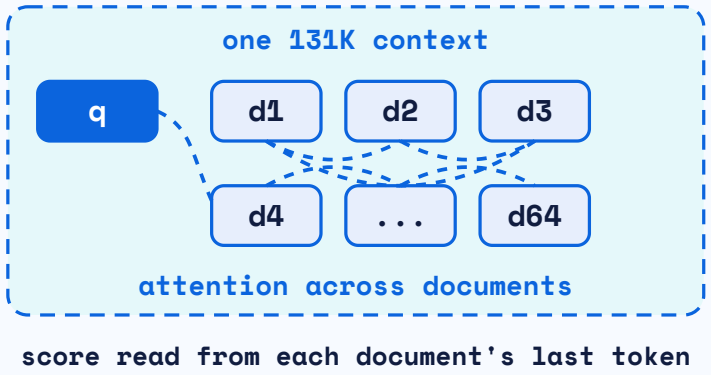
EACH DOCUMENT SCORED ALONE



Every document vector was computed before the other documents existed.

Listwise

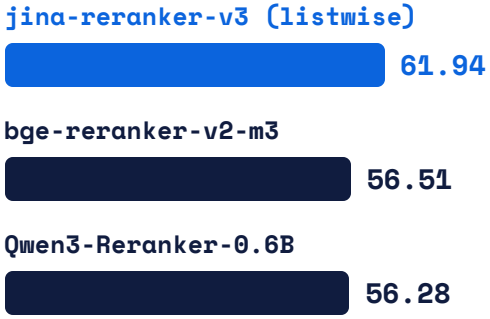
ALL CANDIDATES IN ONE CONTEXT



Each score is read from a document's last token after it has seen every other candidate.

BEIR nDCG@10

0.6B CLASS



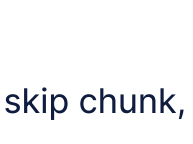
Same size. The gap comes from letting candidates see each other.

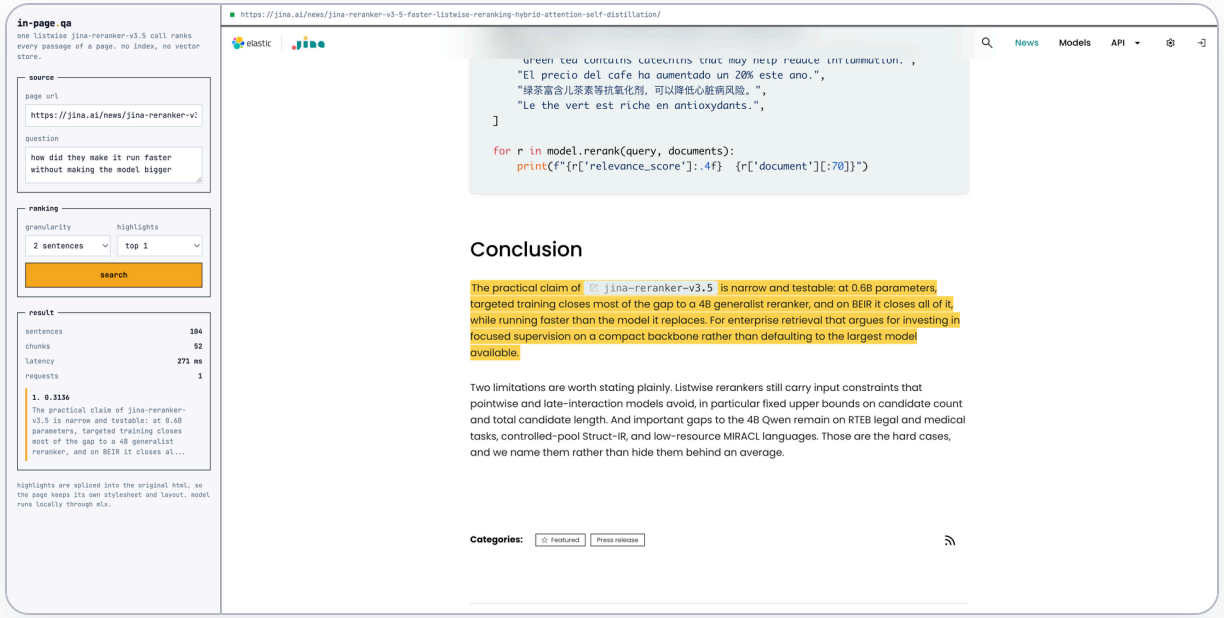
For a single page, drop the index

An index pays off over many queries. A page lives for one visit, so skip chunk, embed and index, and run one listwise pass over the whole page.

GITHUB

- No index, no vector store, no chunk database.
- jina-reranker-v3.5 scores a 53-chunk page in **143 ms** per call, against **4,194 ms** for jina-reranker-v2-base-multilingual.
- Answers are highlighted in place, and the page keeps its own layout.





04

An agent builds the pipeline

Test-time compute is the pattern for long-horizon agents

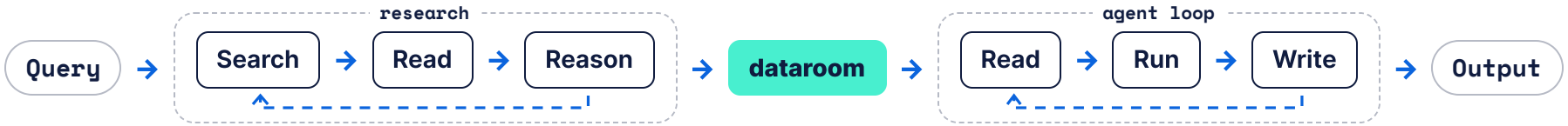
Research and execution want different tokens: build the corpus with cheap local ones, save frontier tokens for the work that needs them.

2025 · Deep research



one web-bound loop · all web IO

2026 · Long-horizon tasks

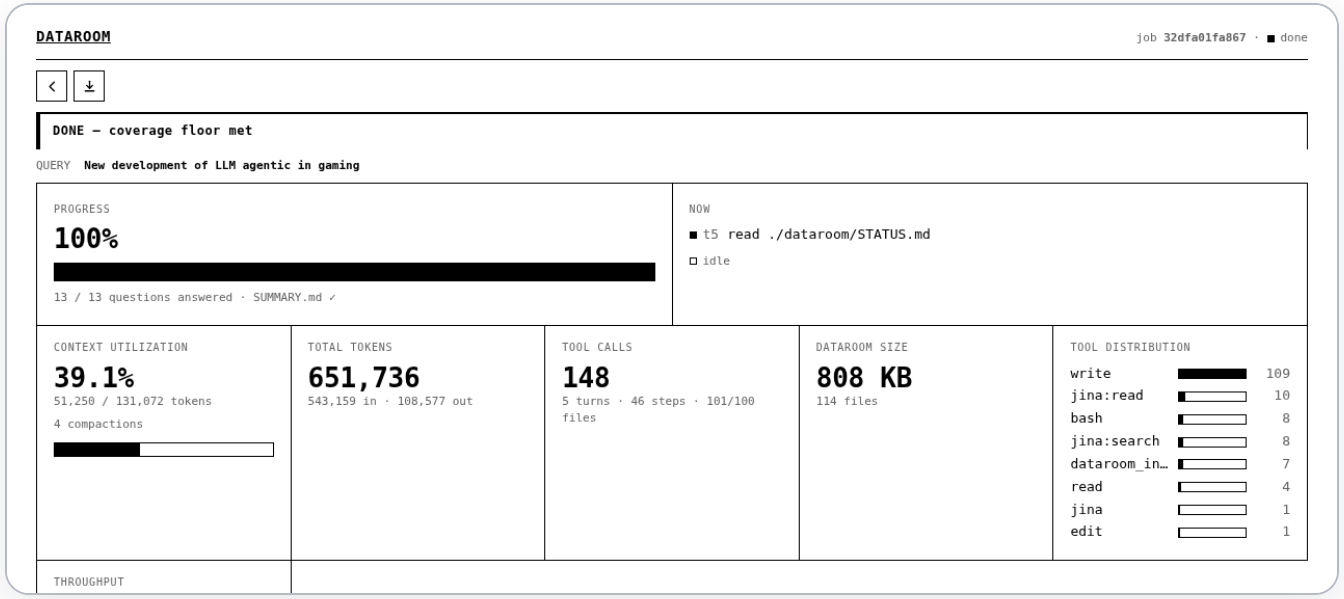


phase 1 research · web IO → build the corpus → phase 2 runs offline · no web IO

dataroom: spend a token budget on building a corpus

A local model runs search, read and write in a loop until the knowledge sits in one cited .zip. It stops at a measured coverage floor.

GITHUB

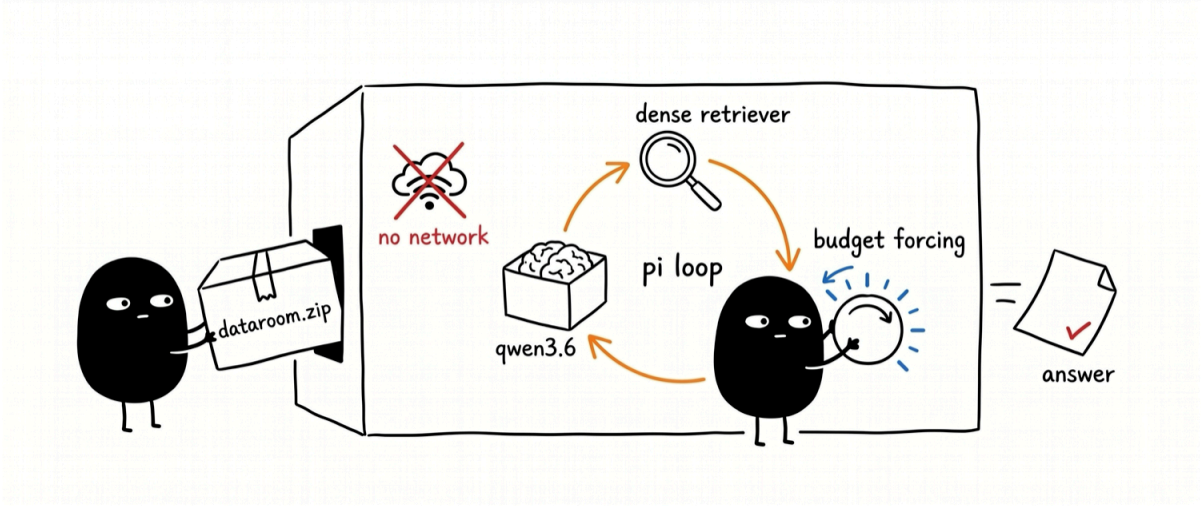


- Stops on outcome, not on a fixed token count.
- The live job page shows coverage, token usage and the tool-call distribution.
- The .zip goes to searchbox or to a frontier model.

searchbox: the agent assembles its pipeline per query

Lock an agent in with one dataroom and no web. It can only answer by composing its own pipeline from local tools.

GITHUB

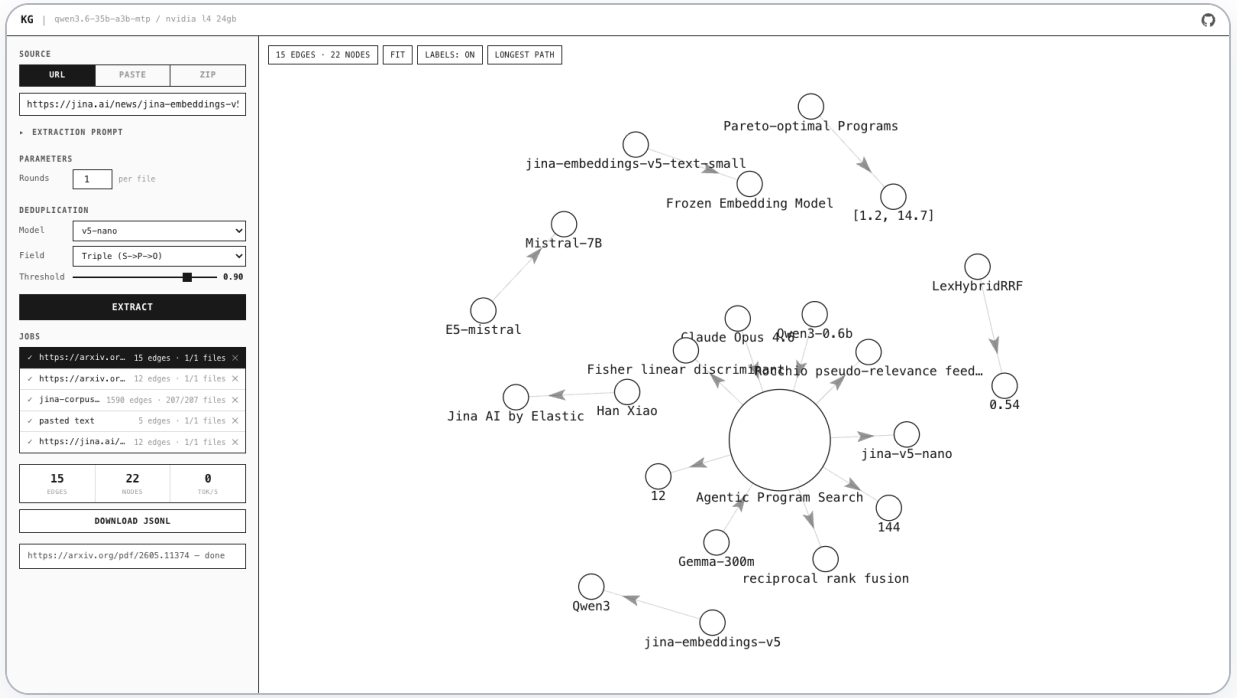


- **Tools:** grep, embed, rerank, similarity, cluster, select_diverse.
- → Which tool does the agent reach for first?
- → Where does a dense retriever add nothing over grep?
- → Does a larger token budget help on the hard questions?

knowledge-graph: hard questions as a private verifier

If one grep finds the answer, every method scores the same. Agentic search needs questions that force several hops.

GITHUB



- Turn the corpus into subject-predicate-object edges.
- Walk paths of length 2 and 3, hide the bridge entities, ask for the endpoint.
- Keep a question only if a closed-book probe fails and a retrieval-equipped solver succeeds.

What the four demos share

Same frozen models throughout. Only the inference budget, and where it is spent, changes.

Training-free

Programs, tags, listwise ranking and agent pipelines all run on frozen weights.

Compute buys quality

+0.24 in-search from programs. A tagger from a retrieval model. A whole index replaced by one call.

New information converts

Patches: +0.37 mAP for free. Crops: +0.075 for 14x. Calibration: about zero.

Check out of domain

+0.24 in search became −0.016 on held-out tasks. Cheap structure and private verifiers survive the check.

Where to spend it in an Elasticsearch stack

Each lever maps to something you can configure today.

RETRIEVER · RRF

Fuse rankings

Combine lexical and dense results with the `rrf` retriever. RRF is what the program search rediscovered.

RETRIEVER · TEXT_SIMILARITY_RERANKER

Rerank the top-k

Add a listwise reranker with the `text_similarity_reranker` retriever and let the candidates see each other.

FIELD · SEMANTIC_TEXT

Score chunks

With `semantic_text` chunking, rank documents by their best chunk instead of one pooled vector.

AGENTS · BUDGET

Budget the agent

Give agents a token budget and a private multi-hop eval before giving them more tokens.

Search has always been test-time compute. Spend it where new information enters.



Thank you

arXiv:2605.11374 · hanxiao.io · x.com/hxiao

Han Xiao, VP of AI, Elastic

THESE SLIDES

hanxiao.io/elasticon-sf-2026

